

pa2-gym

[Start Assignment](#)

- Due Sep 29 by 11:59pm
- Points 8
- Submitting a file upload
- File Types pdf

CS272-26fa-pa2: Build a World, Then Learn It

Project Goals

- Learn how to build a custom environment with Gymnasium
- Learn how to code a model-free control agent with eligibility traces, SARSA(λ)

In PA1 the casino was a black box you could only poke at. This time you are on the other side of the wall. You will build the world yourself, write down its rules so that someone else can understand them, and then hand it to a learning agent that knows nothing about what you built.

Task 1: Your Own Gymnasium Environment

Design and implement your own environment in `myenv.py`. It must follow the [Gymnasium API](https://gymnasium.farama.org/api/env/)  (<https://gymnasium.farama.org/api/env/>).

Requirements

Your environment must satisfy all of the following.

1. **It is a real Gymnasium environment.** It subclasses `gymnasium.Env`, implements `reset`, `step`, `render`, and `close`, and is registered so that `gym.make("cs272/YourEnv-v0")` works. It must pass `gymnasium.utils.env_checker.check_env`.
2. **It is tabular.** `observation_space` is `Discrete(n)` with `20 <= n <= 500`, and `action_space` is `Discrete(m)` with `2 <= m <= 10`. Keep it small!
3. **It is stochastic.** The same action in the same state must not always produce the same outcome. A deterministic puzzle is a search problem, not an interesting MDP. All randomness must come from `self.np_random`, never from `random` or bare `numpy.random`.
4. **Episodes end.** At least one state sets `terminated=True`. Pass `max_episode_steps` to `register()` so that a lost agent is truncated instead of looping forever. Do not set `truncated` by hand — let the `TimeLimit` wrapper do it.

5. **Seeding is honored.** `reset(seed=k)` followed by the same actions must produce exactly the same trajectory, every time, in a fresh process.
6. **It renders.** Support `render_mode="ansi"` and return a human-readable string. You will need it for the sample run in your report.
7. **It is learnable, and worth learning.** A tabular agent must reach a clearly better return than a uniform-random agent within 5,000 episodes. If random play already does as well as your trained agent, the environment is too easy or the reward is not telling the agent anything.
8. **It is yours.** It must not be a built-in Gymnasium environment with the serial numbers filed off. FrozenLake, CliffWalking, Taxi and Blackjack with renamed variables will receive no credit for Task 1.

Environment Documentation

Your repo must include a `README.md` documenting the environment, in the style of the [Gymnasium environment pages](https://gymnasium.farama.org/environments/toy_text/frozen_lake/) [↗\(https://gymnasium.farama.org/environments/toy_text/frozen_lake/\)](https://gymnasium.farama.org/environments/toy_text/frozen_lake/). **The documentation is graded.** It must cover:

- The story: what the world is and what the agent is trying to do
- Observation space, including exactly how you encode the state into one integer
- Action space
- Reward structure
- Starting state, termination, and truncation conditions
- The transition noise: what is random, and with what probability
- Arguments to `__init__` and the registered environment id

A reader who has never seen your code should be able to reimplement your environment from this document alone.

Task 2: SARSA(λ)

Implement SARSA(λ) with eligibility traces in `myagent.py`. The agent gets the environment in its constructor and must read its sizes off `env.observation_space.n` and `env.action_space.n`.

Your agent must not know anything about your environment. It may not import `myenv`, index into a grid, or special-case a state number. It will be run against a different environment than yours during grading, and anything that assumes your own world will fail there.

Tabular SARSA(λ) with eligibility traces

One-step SARSA updates only $Q(s_t, a_t)$ with δ_t . But δ_t is also informative about the pairs visited *before* t , since they led here. The eligibility trace decides how much of δ_t each past pair receives.

You can read the textbook section [here](http://incompleteideas.net/book/first/ebook/node77.html) [↗\(http://incompleteideas.net/book/first/ebook/node77.html\)](http://incompleteideas.net/book/first/ebook/node77.html) and watch the legendary lecture by David Silver

[here](https://youtu.be/0g4j2k_Ggc4?si=gue-KJgJfpLs_-h_&t=3216)  [\(https://youtu.be/0g4j2k_Ggc4?si=gue-KJgJfpLs_-h_&t=3216\)](https://youtu.be/0g4j2k_Ggc4?si=gue-KJgJfpLs_-h_&t=3216)



[\(https://youtu.be/0g4j2k_Ggc4?si=gue-KJgJfpLs_-h_&t=3216\)](https://youtu.be/0g4j2k_Ggc4?si=gue-KJgJfpLs_-h_&t=3216)

(53:38 to 1:19:50).

```

Input:  step size alpha in (0, 1], discount gamma in [0, 1],
        trace decay lambda in [0, 1], exploration epsilon
Init:   Q(s, a) <- 1          for all s in S, a in A
        (Q(terminal, .) = 0 and is never updated)

for each episode:
    E(s, a) <- 0              for all s, a          # traces are episodic
    S <- env.reset()
    A <- eps_greedy(Q, S)      # ties broken at random

    repeat for each step of the episode:
        R, S', terminated, truncated <- env.step(A)
        A' <- eps_greedy(Q, S')

        if terminated:                # no next state exists
            delta <- R - Q(S, A)
        else:                          # incl. truncated: S' is real
            delta <- R + gamma * Q(S', A') - Q(S, A)

        E(S, A) <- E(S, A) + 1         # accumulating

        for all s in S, a in A:
            Q(s, a) <- Q(s, a) + alpha * delta * E(s, a)
            E(s, a) <- gamma * lambda * E(s, a)

        S <- S'
        A <- A'
    until terminated or truncated

```

The λ Sweep

This is the part that shows you understood what λ does.

Train your agent for $\lambda \in \{0, 0.3, 0.6, 0.9, 1.0\}$, with at least **5 seeds per λ** , and produce:

1. **A learning-curve plot.** Mean return per episode against episode number, one line per λ , with the spread across seeds shown (a band or error bars). Smooth the curves over a window of ~100 episodes or they will be unreadable.
2. **A table** giving, for each λ : the number of episodes to first reach a target return of your choosing, and the mean final return. State your threshold.

Then answer, in your report: **why** does λ change the picture on your environment the way it does?

Relate it to how far your reward sits from the decisions that earn it. “ $\lambda=0.9$ was fastest” is an observation, not an answer.

Two free correctness checks

Use both of these before you start debugging anything harder.

- $\lambda=0$ must reduce to ordinary one-step SARSA. If it does not, your trace update is wrong.

Getting Started

The template is in the course GitHub repo: [cs272pub-26fa/pa2_template](https://github.com/sjsu-interconnect/cs272pub-26fa/tree/main/pa2_template)  (https://github.com/sjsu-interconnect/cs272pub-26fa/tree/main/pa2_template).

```
pip install -r requirements.txt
```

Use `myagent.py` and `myenv.py` to implement your agent and environment.

Rules

- This assignment is a **PAIR** project assignment. Find a partner and sign up on People tab on Canvas (PA2 Pair). You must not show to or share your code at any time with other groups.
- Both members must commit to the repo, under their own GitHub accounts, with commit messages that say what was changed. The commit history is how you show that each member made a meaningful contribution to the code; a repo where one member did all the committing, or where the whole project lands in a single commit, will be treated as one where the other member did not contribute.
- You are free to restructure the template (add helper functions, split files) as long as the required file names, the class name `SarsaLambdaAgent`, and its constructor signature stay as given, since the grading harness constructs it directly.
- Your agent will be run against an environment you have not seen. It must not import or reference `myenv`.
- Everything must be reproducible. Seed your runs and say in the report what seeds you used. A learning curve that cannot be regenerated is not a result.

Submission Guide

- Push to a **public GitHub repo** (You may use one of the group member's repo.):
 - `myenv.py` — your environment
 - `myagent.py` — your SARSA(λ) agent
 - `myrunner.py` — training, the λ sweep, and the plot
 - `env.md` — the environment documentation described in Task 1
- Submit to **Canvas** a PDF containing:

- A **clickable** link to the repo (make sure it is public)
- The rendered map or state diagram of your environment, and a one-paragraph description of it
- One sample episode of the trained greedy policy, using your `ansi` renderer, with the return
- The λ sweep plot and table
- Your answer to the “why does λ do that here” question
- Code quality, readability, and documentation are a part of the evaluation criteria.